

دانشگاه علم و صنعت

دانشکده کامپیوتر

گزارش پیاده سازی

فراخوانی سیستمی suicide_bomb

در سیستم عامل xv6-riscv

دانشجو:

پارسا سمیعی نصرآبادی

شماره دانشجویی:

۴۰۱۵۲۱۳۱۸

استاد درس:

دکتر صفوی

تاریخ تحویل:

۲۹ خرداد ۱۴۰۵

فهرست مطالب

۱	خلاصه اجرایی	۲
۲	جدول تغییرات فایل ها	۳
۳	ساختارهای داده هسته و مدیریت فرآیند	۳
۱.۳	بلوک کنترل فرآیند (kernel/proc.h)	۳
۲.۳	جلوگیری از ارث بری (kernel/proc.c)	۳
۴	اجرای کنترل کننده فراخوانی سیستمی	۴
۵	منطق تله سخت افزاری و انفجار	۵
۶	نتایج اجرا و آزمون	۶
۱.۶	سناریوی ۱: انفجار موفق	۶
۲.۶	سناریوی ۲: خنثی سازی موفق	۶
۳.۶	سناریوی ۳: عدم ارث بری در fork	۷
۴.۶	سناریوی چهارم:	۷
۵.۶	سناریوی پنجم:	۸
۶.۶	سناریوی ششم:	۸
۷.۶	سناریوی هفتم:	۹
۷	ملاحظات طراحی معماری	۱۰
۸	دسترسی به کد منبع و مخزن پروژه	۱۰

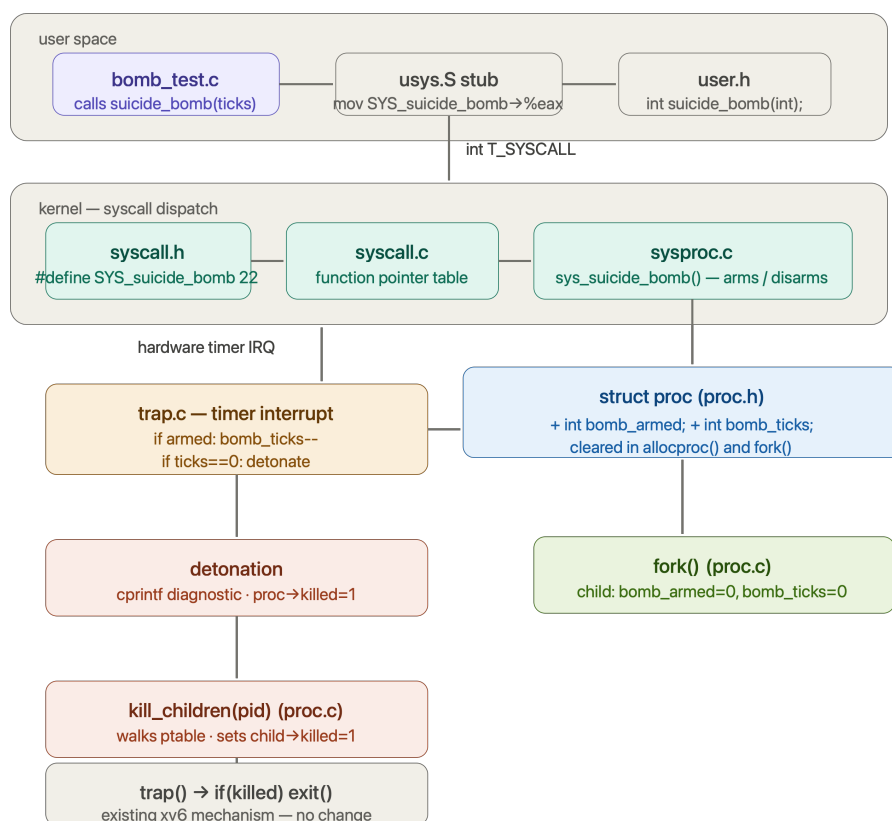
۱ خلاصه اجرایی

این گزارش پیاده‌سازی فراخوانی سیستمی `suicide_bomb(int ticks)` را در سیستم‌عامل xv6-riscv شرح می‌دهد. این ویژگی یک تایمر خودتخریبی با پشتیبانی هسته برای فرایندهای فضای کاربر معرفی می‌کند.

مکانیزم کلی

وقتی یک فرایند تایمر را با تعداد مشخصی تیک فعال می‌کند، هسته در هر وقفه تایمر سخت‌افزاری این شمارنده را کاهش می‌دهد. اگر شمارنده به صفر برسد و پیش از آن غیرفعال نشده باشد، هسته فرایند آغازگر و تمام فرزندان مستقیم آن را خاتمه می‌دهد.

نمودار ۱: معماری suicide_bomb



۲ جدول تغییرات فایل‌ها

فایل	نوع تغییر	توضیحات
kernel/proc.h	تغییر	افزودن فیلدهای bomb_armed و bomb_ticks به struct proc
kernel/proc.c	تغییر	مقداردهی در allocproc، پاکسازی در fork، افزودن تابع kill_children
kernel/sysproc.c	تغییر	پیاده‌سازی کنترل‌کننده sys_suicide_bomb و اعتبارسنجی ورودی
kernel/trap.c	تغییر	پیاده‌سازی منطق شمارش معکوس و انفجار در تله تایمر
kernel/syscall.h	اضافه	ثبت شماره فراخوانی سیستمی جدید
user/bomb_test.c	اضافه	برنامه تست چند-سناریویی در فضای کاربر
user/bombTest.c	اضافه	برنامه تست همراه با یک فرزند

جدول ۱: فهرست فایل‌های تغییر یافته یا ایجاد شده

۳ ساختارهای داده هسته و مدیریت فرآیند

۱.۳ بلوک کنترل فرآیند (kernel/proc.h)

دو فیلد جدید به struct proc اضافه شد تا متغیرهای وضعیت هر فرآیند ایزوله شوند. جداسازی پرچم بولی از شمارنده تیک‌ها، از ابهام بین حالت «غیرفعال» و «فعال با صفر تیک باقیمانده» جلوگیری می‌کند.

```
kernel/proc.h --- struct proc
1 int bomb_armed; // 1 = bomb is active; 0 = disarmed or never
    set
2 int bomb_ticks; // ticks remaining until detonation
```

۲.۳ جلوگیری از ارث‌بری (kernel/proc.c)

فرآیندهای فرزند نباید وضعیت بمب والد خود را به ارث ببرند. در هنگام fork()، متغیرهای فرزند به صراحت خنثی می‌شوند:

kernel/proc.c --- fork()

```

1 np->bomb_armed = 0;
2 np->bomb_ticks = 0;

```

۴ اجرای کنترل‌کننده فراخوانی سیستمی

کنترل‌کننده در kernel/sysproc.c وظیفه انتقال امن آرگومان‌ها از فضای کاربر و اعتبارسنجی را بر عهده دارد.

kernel/sysproc.c --- sys_suicide_bomb

```

1 uint64 sys_suicide_bomb(void) {
2     int ticks_arg;
3     struct proc *p = myproc();
4
5     if(argint(0, &ticks_arg) < 0) return -1;
6     if(ticks_arg < 0) return -1;
7
8     if(ticks_arg == 0){
9         acquire(&p->lock);
10        p->bomb_armed = 0;
11        p->bomb_ticks = 0;
12        release(&p->lock);
13        return 0;
14    }
15
16    acquire(&p->lock);
17    p->bomb_ticks = ticks_arg;
18    p->bomb_armed = 1;
19    release(&p->lock);
20

```

```

21     return 0;
22 }

```

۵ منطق تله سخت‌افزاری و انفجار

در RISC-V، وقفه‌های تایمر در حالت supervisor مدیریت می‌شوند. مکانیسم شمارش معکوس در تابع `usertrap()` قرار دارد.

kernel/trap.c ---

```

1  if(which_dev == 2) {
2      struct proc *p = myproc();
3      if(p && p->state == RUNNING) {
4          acquire(&p->lock);
5          if(p->bomb_armed) {
6              p->bomb_ticks--;
7              if(p->bomb_ticks <= 0) {
8                  printf("suicide_bomb: process %d timed out\n", p->pid);
9                  p->bomb_armed = 0;
10                 p->bomb_ticks = 0;
11                 p->killed = 1;
12
13                 // Release lock before cascading lookup to avoid
14                 // deadlock
15                 release(&p->lock);
16                 kill_children(p->pid);
17                 acquire(&p->lock);
18             }
19             release(&p->lock);
20         }
21     }
22 }

```

```

21     yield();
22 }

```

۶ نتایج اجرا و آزمون

۱.۶ سناریوی ۱: انفجار موفق

در این سناریو، فرآیند تایمر را فعال می کند و پیش از اتمام زمان آن را خاموش نمی کند:

سناریو اول

```

xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2425
cat        2 3 38224
echo       2 4 37000
forktest   2 5 18280
grep       2 6 41776
init       2 7 37384
kill       2 8 36936
ln         2 9 36728
ls         2 10 40352
mkdir      2 11 36992
rm         2 12 36976
sh         2 13 60936
stressfs   2 14 37856
usertests  2 15 193120
grind      2 16 53488
wc         2 17 39248
zombie     2 18 36256
logstress  2 19 39008
forphan    2 20 37760
dorphan    2 21 37192
test       2 22 36264
bomb_test  2 23 46216
bomb       2 24 37320
console    3 25 0
$ bomb_test a

=== Scenario A: Timeout ===
Arming bomb for 50 ticks, then entering infinite loop...
Bomb armed. Spinning. Kernel should terminate this process.

[KERNEL WARNING] Process 4 ('bomb_test') self-destruct in 40 ticks...

[KERNEL WARNING] Process 4 ('bomb_test') self-destruct in 30 ticks...

[KERNEL WARNING] Process 4 ('bomb_test') self-destruct in 20 ticks...

[KERNEL WARNING] Process 4 ('bomb_test') self-destruct in 10 ticks...

=====
[suicide_bomb] DETONATION! Process 4 ('bomb_test') TERMINATED
=====
$ █

```

۲.۶ سناریوی ۲: خنثی سازی موفق

در این سناریو، فرآیند تایمر را پیش از انقضا با `suicide_bomb(0)` خنثی می کند:

سناریو دوم

```
$ bomb_test b

=== Scenario B: Safe execution ===
Arming bomb for 200 ticks...
Bomb armed. Performing quick computation...
Computation done (result=1783293664). Disarming bomb...
Bomb disarmed safely. Exiting cleanly.

$
```

۳.۶ سناریوی ۳: عدم ارث‌بری در fork

این سناریو تأیید می‌کند که فرزند، وضعیت بمب والد را به ارث نمی‌برد:

سناریو سوم

```
Usage: bomb_test [a|b|neg|double|fork|multi]
$ bomb_test fork

--- Edge Case 2: Fork Inheritance Test ---
Parent arming bomb for 150 ticks...
Child born. Waiting briefly to see if parent's bomb blows it up...
SUCCESS: Child survived! Wiping slate clean worked. Disarming child.
Parent spinning until detonation...

[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 140 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 130 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 120 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 110 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 100 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 90 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 80 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 70 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 60 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 50 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 40 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 30 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 20 ticks...
[KERNEL WARNING] Process 9 ('bomb_test') self-destruct in 10 ticks...

=====
[suicide_bomb] DETONATION! Process 9 ('bomb_test') TERMINATED
=====

$
```

۴.۶ سناریوی چهارم:

این سناریو تأیید می‌کند که امکان اجرای سیستم‌کال با عدد منفی وجود ندارد:

سناریو چهارم

```
$ bomb_test neg

=== Error path: negative ticks ===
Bomb armed with 100 ticks.
suicide_bomb(-5) returned -1 (expected -1)
Disarmed. Exiting.

$
```

۵.۶ سناریوی پنجم:

این سناریو تأیید می‌کند که امکان اجرای سیستم‌کال با عدد بزرگ‌تر از `int` وجود ندارد:

سناریو پنجم

```
$ bomb_test double

--- Edge Case 1: Double-Arming Test ---
Arming bomb with 500 ticks...
Overriding bomb with 20 ticks (should shorten countdown)...
Spinning. Process should detonate quickly (~20 ticks), not 500.

[KERNEL WARNING] Process 14 ('bomb_test') self-destruct in 10 ticks...

=====
[suicide_bomb] DETONATION! Process 14 ('bomb_test') TERMINATED
=====

$
```

۶.۶ سناریوی ششم:

این سناریو تأیید می‌کند که امکان اجرای سیستم‌کال در چند پراسس هم‌زمان وجود دارد:

سناریو ششم

```

$ bomb_test multi

--- Edge Case 3: Multi-Process Stress Test ---
Spawning 3 concurrent armed processes to check kernel lock stability...
ChChCilhd 2ilild 24 3d a 22 armaerd medform efor rd30 f 5tori0 t 40 ticickcs
scks

[KERNEL WARNING] Process 24 ('bomb_test') self-destruct in 40 ticks...
[KERNEL WARNING] Process 22 ('bomb_test') self-destruct in 20 ticks...
[KERNEL WARNING] Process 23 ('bomb_test') self-destruct in 30 ticks...
[KERNEL WARNING] Process 22 ('bomb_test') self-destruct in 10 ticks...
[KERNEL WARNING] Process 24 ('bomb_test') self-destruct in 30 ticks...
[KERNEL WARNING] Process 23 ('bomb_test') self-destruct in 20 ticks...

=====
[suicide_bomb] DETONATION! Process 22 ('bomb_test') TERMINATED
=====

[KERNEL WARNING] Process 24 ('bomb_test') self-destruct in 20 ticks...
[KERNEL WARNING] Process 23 ('bomb_test') self-destruct in 10 ticks...
[KERNEL WARNING] Process 24 ('bomb_test') self-destruct in 10 ticks...

=====
[suicide_bomb] DETONATION! Process 23 ('bomb_test') TERMINATED
=====

=====
[suicide_bomb] DETONATION! Process 24 ('bomb_test') TERMINATED
=====

SUCCESS: All children detonated without kernel panicking!
$ 

```

۷.۶ سناریوی هفتم:

اجرای این دو کد امکان بررسی این سیستم کال در شرایط متفاوت را نشان می دهند:

سناریو هفتم

```

$ test
Testing suicide_bomb syscall...
Bomb armed for 30 ticks. Entering infinite loop...

[KERNEL WARNING] Process 26 ('test') self-destruct in 20 ticks...
[KERNEL WARNING] Process 26 ('test') self-destruct in 10 ticks...

=====
[suicide_bomb] DETONATION! Process 26 ('test') TERMINATED
=====

$ bomb

--- STARTING SUICIDE BOMB TEST ---
Parent: My PID is 27
Child: I am alive! My PID is 28
Parent: Arming the bomb for 50 ticks...

[KERNEL WARNING] Process 27 ('bomb') self-destruct in 40 ticks...
[KERNEL WARNING] Process 27 ('bomb') self-destruct in 30 ticks...
[KERNEL WARNING] Process 27 ('bomb') self-destruct in 20 ticks...
[KERNEL WARNING] Process 27 ('bomb') self-destruct in 10 ticks...

=====
[suicide_bomb] DETONATION! Process 27 ('bomb') TERMINATED
=====

$ 

```

۷ ملاحظات طراحی معماری

نکته ۱: جلوگیری از بن بست ABBA

قفل فرآیند فراخواننده ($p \rightarrow \text{lock}$) باید پیش از اجرای `kill_children()` آزاد شود تا از بن بست کلاسیک ABBA جلوگیری شود. `kill_children` برای یافتن فرزندان باید قفل های دیگر فرآیندها را بگیرد.

نکته ۲: خاتمه تأخیری (Deferred Termination)

از اجرای مستقیم `exit()` در بستر وقفه خودداری شده است. پرچم `p->killed = 1` خاتمه برنامه را به صورت ایمن به لایه های استاندارد بازگشت به محیط محول می کند و از ناسازگاری با پشته وقفه جلوگیری می شود.

- ایزولاسیون وضعیت: جداسازی `bomb_armed` از `bomb_ticks` ابهام بین «غیرفعال» و «صفر تیک» را حذف می کند.
- صحت همروندی: تمام دسترسی ها به متغیرهای بمب زیر قفل `p->lock` انجام می شوند.
- ارث بری صفر: خنثی سازی صریح در `fork` از انتشار ناخواسته وضعیت بمب جلوگیری می کند.

۸ دسترسی به کد منبع و مخزن پروژه

تمامی کدهای پیاده سازی شده، فایل های پیکربندی و تاریخچه تغییرات (commit history) این پروژه به صورت متن باز در پلتفرم گیت هاب میزبانی می شود. برای کلون کردن مخزن یا مشاهده آنلاین کدهای منبع می توانید از کادر زیر استفاده کنید:

لینک مخزن گیت هاب پروژه



لینک دسترسی آنلاین در گیت هاب:

github.com/ParsaSamiei/xv6-os-suicide-bomb-syscall